

Python RegEx

Regular expressions in Python live in the `re` module. Always write patterns as raw strings (`r"..."`) so backslashes reach the regex engine instead of being eaten by Python's string parser.

The re Module

Call	Returns / does
<code>re.search(p, s)</code>	first match anywhere, else <code>None</code>
<code>re.match(p, s)</code>	match only at the start of the string
<code>re.fullmatch(p, s)</code>	match only if the whole string matches
<code>re.findall(p, s)</code>	list of all matches (or group tuples)
<code>re.finditer(p, s)</code>	iterator of match objects
<code>re.sub(p, repl, s)</code>	replace all matches
<code>re.split(p, s)</code>	split the string on the pattern
<code>re.compile(p)</code>	pre-compile a pattern for reuse

Extract, then use the match

```
import re

m = re.search(r"(\d{4})-(\d{2})-(\d{2})", text)
if m:
    year, month, day = m.groups()
    print(m.group(0))    # the full match
```

Character Classes

Token	Matches
<code>.</code>	any character except newline
<code>\d / \D</code>	a digit / a non-digit
<code>\w / \W</code>	word char (a-z, 0-9, _) / non-word
<code>\s / \S</code>	whitespace / non-whitespace
<code>\b</code>	a word boundary
<code>[abc]</code>	any one of a, b, c
<code>[^abc]</code>	any char except a, b, c
<code>[a-z0-9]</code>	a range: lowercase letter or digit

Anchors & Quantifiers

Token	Meaning
<code>^ / \$</code>	start / end of string (or line with <code>re.M</code>)
<code>*</code>	0 or more (greedy)
<code>+</code>	1 or more
<code>?</code>	0 or 1 (optional)
<code>{3}</code>	exactly 3
<code>{2,5}</code>	between 2 and 5
<code>{2,}</code>	2 or more
<code>*? +? ??</code>	lazy — match as few as possible

Greedy by default. `.*` grabs as much as it can and backtracks. For `<.*>` on `<a><b>` that's the whole string — use the lazy `<.*?>` to stop at the first `>`.

Groups & Lookarounds

Token	Meaning
<code>(abc)</code>	capturing group — numbered from 1
<code>(?:abc)</code>	group without capturing
<code>(?P<name>abc)</code>	named group — read via <code>m.group('name')</code>
<code>`a`</code>	<code>b`</code>
<code>(?=abc)</code>	lookahead: followed by abc
<code>(?!abc)</code>	negative lookahead: not followed by abc
<code>(?<=abc)</code>	lookbehind: preceded by abc
<code>\1</code>	backreference to group 1

Named groups + sub with a backreference

```
# swap "First Last" → "Last, First"
re.sub(r"(?P<f>\w+)\s+(?P<l>\w+)",
       r"\g<l>, \g<f>",
       "Grace Hopper")
# → "Hopper, Grace"
```

Flags

Flag	Effect
<code>re.I (IGNORECASE)</code>	case-insensitive matching
<code>re.M (MULTILINE)</code>	<code>^</code> and <code>\$</code> match at each line
<code>re.S (DOTALL)</code>	<code>.</code> also matches newlines
<code>re.X (VERBOSE)</code>	ignore whitespace/comments in the pattern

Combine flags with |

```
re.findall(r"^error:.*", log, re.I | re.M)
```